

理论计算机科学导引

Lecture Notes

By Huijiao.



Chapter 1

引子.

function
specification
what?

 $\Leftrightarrow$

Program
implementation
how

定义1. Binary String.

字符集 $\Sigma = \{0, 1\}$. (可推广)

$$\Sigma^n = \{(a_1, \dots, a_n) \mid a_i \in \Sigma\}.$$

↓

Binary String

$\Sigma^0 = \{\epsilon\}$. 空串.

$$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \dots = \bigcup_{n \geq 0} \Sigma^n$$

x 为 y 前缀: $y = xz$. $\exists z$.

定义2: Encoding.

$E: A \rightarrow \{0, 1\}^*$. 单射.

例2.1: $N \times N \rightarrow \{0, 1\}^*$.

$(a, b) \rightarrow E(a)E(b) \quad \times \quad (E(1,6) = E(3,2)).$
prefix 冲突.

$\therefore \exists \lambda$ prefix-free.

定义3. prefix-free Encoding

$E(x)$ 不为 $E(x')$ 前缀, $\forall x \neq x'$

有: $E: A \rightarrow \{0,1\}^*$, prefix-free

欲得: $\bar{E}: A^* \rightarrow \{0,1\}^*$, one-to-one.

思考: $(a_1, \dots, a_n) \in A^*$

令: $\bar{E}(a_1, \dots, a_n) = \begin{cases} E(a_1) \dots E(a_n), & n \neq 0 \\ \epsilon, & n = 0 \end{cases}$
(自然定义).

为什么对?

引理1: $\bar{E}$ 为单射.

证: 若 $\exists (a_1, \dots, a_p) \neq (b_1, \dots, b_q).$

但 $E(a \dots) = E(b \dots).$

find first $i : a_i \neq b_i$

则 $E(a_1) \dots E(a_p) = E(b_1) \dots E(b_q)$ ($a_1 \sim a_{i-1}$ 已匹配).

$\therefore E(a_i)$ 与 $E(b_i)$ 有前缀关系.

矛盾!

#.

引理 2.

若 $\exists$ encoding $E: A \rightarrow \{0,1\}^*$

则 $\exists$ prefix-free $E': A \rightarrow \{0,1\}^*$.

s.t. $|E'(a)| \leq 2|E(a)| + 2, \forall a$.

证:

$E(a) = 010, E'(a) = 001100 + 01$ (显然 one-to-one)

此时: 若 $E(x)$ 为 $E(y)$ 前缀, 则 $\text{len}(E(x)) \leq \text{len}(E(y))$.

若 $\text{len}(E(x)) < \text{len}(E(y))$, 必有 "01" 匹配 "00"/"11". 矛盾.

$\therefore \text{len}(E(x)) = \text{len}(E(y))$

$\therefore E(x) = E(y), x = y$.

#.

思考: $|E'(a)| \leq |E(a)| + O(\log |E(a)|)$. ?

定理 1:

若 $\exists$ one-to-one $E: A \rightarrow \{0,1\}^*$.

则 $\exists$ one-to-one $E': A^* \rightarrow \{0,1\}^*$.

证: 由引理 1. 2. 立得.

#.

定义 4. 可数. (至多可数).

(1). 有限

(2). A 与 N 双射 $\Downarrow$.

(3). $\exists$ one-to-one $f: A \rightarrow N$.

(4). $\exists$ onto func. $g: N \rightarrow A$.

引理 3: $\{0,1\}^*$ 可数.

(有限集的可列并可数).

定理 2: Countable $\Leftrightarrow$ 可 Encoding.

(比较 trivial 略了...)

定义 5. Problem

$$f: \{0,1\}^* \rightarrow \{0,1\}^*$$

定理3: Problem is Uncountable.

Cantor 对角化.

(lec2).

Chapter 2 : Bool 电路

boolean Circuit.

基本单元: and, or, not.

例1: $\text{Xor}(a, b) = (a \wedge b) \vee (a \vee b)$.

一个 bool 电路 C

↓ 抽象

无权. 有向图.

n input nodes
 $x_0 \sim x_{n-1}$

(no in-arc, ≥ 1 out-arc)

S gates: 1 out-arc

↓
 m output nodes
 $y_0 \sim y_{m-1}$

一个电路 C.

$|C| = S.$

定义1: C computers $f: \{0,1\}^n \rightarrow \{0,1\}^m$

$\Leftrightarrow$ 对于任意输入, 输出对应.

定义2: AON-Circ Program.

略

定理1: f 可被电路计算 $\Leftrightarrow$ 可被 AON-Circ 计算.

此时, 行数 = 门数.

下面来看与非门: $NAND(a,b) = \text{not}(\text{and}(a,b))$.

and, not, or 可用 NAND 实现.

NAND circuit $\longrightarrow$ Boolean circuit

$s \quad \Rightarrow \quad \leq 2s$

$\leq 3t \quad \Leftarrow \quad t$

定理2: $\forall n, m \geq 0$ 及 $f: \{0,1\}^n \rightarrow \{0,1\}^m$

$\exists$ 一个 boolean circuit 计算它.

问: 至少要多少个门?

粗略上界: $2^n \cdot n \cdot m$.

定义3: 一组门可计算上述函数, 称之 Universal.

思考: 如何判断 $F = \{f_1, \dots, f_n\}$ 是否 Universal.

只需判断 F 可否导出 **Not AND** !

回顾定理2, $O(m \cdot n \cdot 2^n)$ 是否有冗余? 是否紧?

定理2 (扩展): 可改进为 $O(\frac{2^n m}{n})$..

证: 用 NAND-CIRC program.

引入语法特性. (sugar)

1. loop of fix length (循环)

2. user-defined-procedure (函数).

3. Conditional statement (条件)

} 均可用 NAND

例2: ADD. $\{0,1\}^{2^n} \rightarrow \{0,1\}^{n+1}$

def ADD

Result = [0] $\times$ (n+1)

$$\text{Carry} = [0] \times (n+1).$$

for i in range (n) :

$$\text{Result}[i] = \text{XOR}(\text{carry}[i], \text{XOR}(x[i], x[i+n]));$$

$$\text{Carry}[i+1] = \text{MAJ}(\text{carry}[i], x[i], x[i+n]).$$

$$\text{Result}[n] = \text{Carry}[n]$$

Return Result

($O(n)$ 行).

MUL: $2n$ 次加法, $O(n^2) \xrightarrow{\text{优化}} O(n^{\log_2 3})$ (略).

例3: Lookup: $\{0, 1\}^{2^k + k} \rightarrow \{0, 1\}.$

$$x \xrightarrow{2^k}$$

$\square_i$: lookup x 的第 i 位.

归纳 (递归).

$$\text{lookup}_1(x_0, x_1, i_0)$$

$$\text{if } i_0 = 0$$

$$y_0 = x_1$$

else

$$y_0 = x_0.$$

(类似 = 进制分治).

$$\text{lookup}_2(x \dots, i_0, i_1)$$

$$\text{if } i_0 = 0$$

$$Y_0 = \text{lookup}_1 (x_2, x_3, i1)$$

else

$$Y_0 = \text{lookup}_1 (x_0, x_1, i1)$$

lookup_k 由 lookup_{k-1} 即可。

$$\begin{cases} L(k) = C + 2L(k-1) \\ L(1) = O(1) \end{cases}$$

$$\Rightarrow L(k) = 2^k$$